

Análise Estrutural e Arquitetura

Este relatório contém o mapa completo de funcionamento do dashboard **DocTrack v3.0**, descrevendo tecnologias, estrutura de diretórios, esquema de banco de dados, e detalhamento de interface.

1. Visão Geral do Sistema

Pilha Tecnológica (Stack)

- **Backend:** Python 3, Flask, SQLAlchemy, JWT (Flask-JWT-Extended).
- **Frontend:** SPA Vanilla (HTML5, CSS3 Variables, JavaScript ES6).
- **Banco de Dados:** SQLite (doctrack.db).
- **Design System:** UI Dark/Cyberpunk (Navy, Cyan, Purple).

Capacidades Principais

- Rastreabilidade Completa de Documentos Industriais.
- Dashboard analítico em tempo real com Chart.js.
- Controle de Acesso em Níveis (Admin, Gestor, Técnico, Leitura).
- Auditoria rigorosa (Log de todas as ações).

2. Arquitetura do Backend (Python)

O servidor Flask está estruturado de forma monolítica mas com clara separação de responsabilidades:

- **servidor.py:** Orquestrador principal. Inicializa o banco de dados e expõe as rotas REST (`/api/data`, `/api/documentos`). Importa os dados da base legada em Excel (`Consolidado_Dashboard_Documentos.xlsx`).
- **models.py:** Define as tabelas usando ORM. Implementa a lógica do *Status Global* dinâmico (calculado em tempo de execução com base nas etapas) e controle de versionamento Otimista para evitar colisão na edição.
- **auth.py:** Blueprint focado em autenticação. Endpoints de Login, Refresh e listagem de usuários com proteção de escopo (`@require_role`).
- **agente_scanner.py:** Sistema auxiliar para buscar, relatar e auditar anomalias em arquivos de rede (ainda operando em formato mock ou leitura direta).

Esquema de Banco de Dados

Tabela	Principais Campos	Responsabilidade
users	email, senha_hash, role, ativo, ultimo_login	Autenticação e controle de privilégios.
documentos	equipamento, origem, etapa_elaboracao, etapa_revisao1...	Armazenar os metadados dos documentos técnicos.
audit_logs	usuario_email, acao, entidade, valor_antigo, valor_novo	Rastreamento de ponta a ponta (Quem alterou o que e quando).

3. Arquitetura do Frontend (SPA)

O frontend foi desenvolvido sem frameworks pesados (No-React/No-Vue), garantindo máxima performance e independência através de renderização dinâmica em JavaScript.

- **dashboard.html**: Único ponto de entrada. Contém a estrutura estática das páginas (Dashboard, Docs, Audit, Users) que são alternadas via classe CSS `.active`. Contém todos os modais de inserção/edição.
- **style.css**: Responsável pelo tema imersivo. Utiliza Variáveis CSS no `:root` para definir a paleta de cores primárias, permitindo fáceis alterações de tema. Aplica layouts em *CSS Grid* e *Flexbox* para responsividade.
- **app.js**: Coração do sistema no cliente. Mantém um cache local de todos os documentos (`allDocs`) viabilizando uma busca quase instantânea. Cuida da atualização automática de gráficos com *Chart.js* e envia as requisições autenticadas usando o header *Bearer Token*.

4. Estrutura de Interface (Telas)

Dashboard (Visão Geral)

Apresenta o painel de KPIs numéricos no topo (Finalizados, Em Progresso, Pendentes). Integra componentes gráficos (Donut, Barras) para demonstrar a saúde atual dos fluxos de aprovação e categorias, além de uma tabela resumo das últimas modificações.

Todos os Docs

Área de listagem profunda. Destaca-se por uma navegação em **Abas Laterais/Horizontais** (Filtro por Origem/Departamento), scroll horizontal fixo (*sticky-column*) para a coluna Equipamento e formulários modais dinâmicos para inserção/alteração.

Auditoria e Configurações

Páginas restritas a perfis com privilégios. A auditoria apresenta todos os *AuditLogs* com campos que diferenciam exatamente o que foi modificado (antes e depois). As configurações permitem reimportar o Excel raiz.

5. Conclusão e Próximos Passos (Melhorias)

O DocTrack v3.0 superou o formato "planilha" e se tornou uma aplicação transacional. Para uma evolução futura (v4.0), recomendam-se os seguintes pontos arquiteturais:

- **Performance** Implementar *Server-Side Pagination* no endpoint `/api/documentos` caso o volume passe de 5.000 itens, reduzindo o *payload* da carga inicial.
- **Banco de Dados** Migrar de SQLite para PostgreSQL, visando lidar com múltiplas transações simultâneas em um ambiente corporativo real.
- **Integração** Ligar o sistema a um repositório de nuvem (AWS S3 ou SharePoint API) para gerenciar o upload dos arquivos físicos diretamente pelo painel.

Relatório gerado automaticamente através da análise de código fonte. Confidencial e de uso interno.